

Deep Learning Practical 1

Jupyter, Colab and NCC setup

2026–27

Practical overview

In this two-hour practical, first set up your environment, then train and improve a small digit classifier. The aims are to:

- Ensure a suitable deep learning training environment.
- Chat with the staff about any questions about the module.

Environment setup

It is important that, after this practical, everyone has an environment setup. There are generally four ways to run the code and train the large deep learning models required for this module:

- Use the NCC Jupyter server: this is recommended for most students, although sometimes I/O can be a little slow.
- Use Google Colab: this is easy to get started, but the usage gets throttled and the kernel is not persistent, so only use it for a backup.
- SSH to NCC and use SLURM: this is only recommended for those willing to take time to learn SLURM.
- Use the lab machines or your home machines: having a good home GPU won't offer any advantage.

Using Google Colab

- You need a Google account for this
- Simply open an existing .ipynb in Colab and run the cells—here are two to get started with:
 - Train a classifier: [📄 download](#)
 - Train a convnet: [📄 download](#)
- Check that your 'Runtime > Change runtime type' is a GPU

NCC usage via Jupyter (recommended)

- Connect to a machine from the university network, otherwise use the VPN to connect from outside
- Login to <https://ncc1.clients.dur.ac.uk/COMP0000> with your CIS username and password (if you get an authentication failure, you might not have an account on NCC and need to email Rob Powell)
- Select appropriate preferences for the job, e.g. `ug-gpu-small` with 1 GPU
- Once it takes you to your home directory click “Assignments” tab and “Fetch” the environment creation folder
- Click the title of the folder “Create Jupyter Environment Script” and click the “CreateJupyterPythonEnvironment” script that is inside it
- In the second code cell you need to enter a Python version, and two names (these can be the same if you like). The `virtualEnvName` becomes the directory name of the `virtualenv` in your NCC home directory. The `jupKernelName` is what appears in the Jupyter menu later. If you're only using PyTorch you shouldn't need modules as CUDA is bundled

- Run the whole notebook (Kernel > Restart and Run All). Once it has finished (about 4-5 mins) you can close this tab and return to the previous tab that should still be on the Assignments page.
- Click “Files” to see home directory again and then refresh the page otherwise the new kernel won’t appear in the list!
- Click “New” and select your kernel name from the dropdown list
- Run your `!pip install` commands (only needs to be done the first time you ever use the kernel, unlike Colab, as our storage is persistent)
- Download, upload to NCC and run the classifier.ipynb [↗](#) download and convnet.ipynb examples [↗](#) download

NCC usage via SSH

Only do this if you know what you’re doing, e.g. have used SLURM before:

- SSH to the head node `CIS-username@ncc1.clients.dur.ac.uk` (either via VPN or Mira)
- Do not run code on this head node. Doing so will cause problems for all other users, can interrupt active postgraduate research and may get you banned
- Before running any code, read the NCC documentation carefully: <https://ncc1.clients.dur.ac.uk> and do your compute with SLURM
- We recommend using `tmux` for a persistent shell along with `zsh`
- Please respect other users in the queue

PyTorch (and optional JAX) preliminaries

After completing the classifier below, you can use your new environment to explore the PyTorch and optional JAX preliminaries from the “Dive into Deep Learning” book:

https://d2l.ai/chapter_preliminaries/ndarray.html

- Work your way from section 2.1 (Data Manipulation) to the end of section 2.3 (Linear Algebra). If you have any questions, please ask the instructors.
- We would recommend reading ahead in this book and practicing the examples in both PyTorch and JAX. How does JAX differ from PyTorch in these examples?
- You may wish to also take time now to integrate your workflow with `vscode` or `cursor`, and discuss best/recent practices with the instructors and those around you.

Open the supplied practicals

Open `dl-practical1.py` [↗](#) in VS Code or IPython, using the environment created above. Run its sections in order. Each practical’s Python file includes the code it needs and downloads its dataset on first use. Copy code from the Python file: PDF text extraction can change quotes, minus signs and indentation.

For the assessed exercises in Practicals 5–9, download the corresponding `dl-practicalN.py`. Each file contains its training, scoring and ZIP export code. Run the complete file or paste the complete program into one notebook cell.

Select the kernel you created above. Run the following cell to check the environment:

```
import sys
import torch
import torchvision

print(sys.executable)
print("torch:", torch.__version__)
print("torchvision:", torchvision.__version__)
print("CUDA available:", torch.cuda.is_available())
device = "cuda" if torch.cuda.is_available() else "cpu"
x = torch.tensor([1., 2., 3.], device=device)
print(x, x.device, (x * x).sum().item())
```

The calculation prints 14. If CUDA is available, the tensor is on the GPU. A notebook and terminal can use different Python environments: compare `sys.executable` if an import works in one but fails in the other.

1 Train a first digit classifier

Once the environment works, use it to train a small network. Work through the following cells in `dl-practical1.py`. The aim is to understand what changes during training and to make a measurable improvement yourself.

Images, labels and batches

An MNIST example is a 28×28 greyscale image and a label from 0 to 9. The following code downloads the images and returns them with a channel dimension, with pixel values scaled to $[-1, 1]$. A batch of 64 therefore has shape $(64, 1, 28, 28)$; its labels have shape $(64,)$. The loader uses 50,000 examples for training and reserves 10,000 different examples for validation.

```
import torch
from torch import nn
from torch.nn import functional as F
from torchvision.datasets import MNIST
import matplotlib.pyplot as plt

def load_data():
    dataset = MNIST("data", train=True, download=True)
    order = torch.randperm(60000, generator=torch.Generator().manual_seed(3801))
    images = dataset.data[order].unsqueeze(1).float()/127.5 - 1
    labels = dataset.targets[order]
    return images[:50000], labels[:50000], images[50000:], labels[50000:]

images, labels, validation_images, validation_labels = load_data()
print(images.shape, labels.shape)
plt.imshow(images[0, 0], cmap="gray", vmin=-1, vmax=1)
plt.title(f"Label: {labels[0].item()}")
plt.axis("off")
plt.show()
```

Exercise

Display a different example. Find its brightest and darkest pixels with `min()` and `max()`. What does `images[:64].shape` return? Which axis selects the image, and which axes select a pixel?

Turn pixels into ten scores

Flattening an image gives a vector $x \in \mathbb{R}^{784}$. Our network has 16 hidden units:

$$h = \text{ReLU}(W_1 x + b_1), \quad z = W_2 h + b_2, \quad \text{ReLU}(a) = \max(0, a).$$

The ten entries of z are scores, called logits. The predicted digit is the index of the largest score. The matrices have shapes 16×784 and 10×16 ; the biases have 16 and 10 entries.

```
def make_model(width=16):
    return nn.Sequential(nn.Flatten(), nn.Linear(784, width),
                        nn.ReLU(), nn.Linear(width, 10))

torch.manual_seed(3801)
device = "cuda" if torch.cuda.is_available() else "cpu"
model = make_model().to(device)
x, y = images[:64].to(device), labels[:64].to(device)
logits = model(x)
print(logits.shape)
print(logits.argmax(dim=1)[:10], y[:10])
print(sum(p.numel() for p in model.parameters()))
```

Exercise

Explain why the output has shape (64, 10). Count the weights and biases by hand. Before training, should these predictions agree with the labels?

2 A loss and a gradient update

Softmax converts the ten logits into probabilities:

$$p_k = \frac{e^{z_k}}{\sum_{j=0}^9 e^{z_j}}, \quad \ell(z, y) = -\log p_y.$$

If the correct digit receives probability 0.1, the loss is about 2.303; if it receives probability 0.9, the loss is about 0.105. Training tries to reduce this loss over many examples. `F.cross_entropy` combines softmax and the logarithm numerically stably and averages over the batch. Give it the logits directly.

```
loss = F.cross_entropy(logits, y)
probabilities = logits.softmax(dim=1)
print(loss.item(), probabilities[0], probabilities[0].sum().item())
```

Exercise

For the first image, inspect the probability assigned to its true label and compare `-probabilities[0, y[0]].log()` with the batch loss. Why are they generally different?

The gradient tells us how the loss changes when each parameter changes. Plain gradient descent would use $\theta \leftarrow \theta - \eta \nabla_{\theta} L$. Here we use Adam, which adjusts the update using running estimates of the gradients. We will study optimisers in more detail later.

```
def train(model, images, labels, device, steps=150, learning_rate=0.003):
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    history = []
    model.train()
    for step in range(steps):
        index = torch.randint(len(images), (64,))
        x, y = images[index].to(device), labels[index].to(device)
        logits = model(x)
        loss = F.cross_entropy(logits, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        history.append(loss.item())
        if (step + 1) % 25 == 0:
            print(step + 1, round(loss.item(), 3))
    return history
```

```
history = train(model, images, labels, device)
plt.plot(range(1, len(history)+1), history)
plt.xlabel("Gradient update")
plt.ylabel("Batch cross-entropy")
plt.show()
```

Exercise

Match each line of the loop to one operation: choose a batch, predict, measure error, clear old gradients, differentiate, update parameters. Why do the losses sometimes rise between updates?

3 Check the predictions and improve them

Validation examples are used to measure performance and are never passed to the optimiser. Accuracy is the fraction whose predicted label is correct:

$$\text{accuracy} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}\{\arg \max_k f_{\theta}(x_i)_k = y_i\}.$$

```
@torch.no_grad()
def accuracy(model, images, labels, device):
    model.eval()
    correct = 0
    for x, y in zip(images.split(256), labels.split(256)):
        correct += (model(x.to(device)).argmax(1).cpu() == y).sum().item()
    return correct / len(images)
```

```
score = accuracy(model, validation_images, validation_labels, device)
print(f"Validation accuracy: {100*score:.2f}%")
```

The supplied 16-unit model obtained 84.52% validation accuracy after 150 updates on the GTX 1080 Ti, using seed 3801, batches of 64 and learning rate 0.003. These figures show the actual run; small numerical differences between environments are expected.

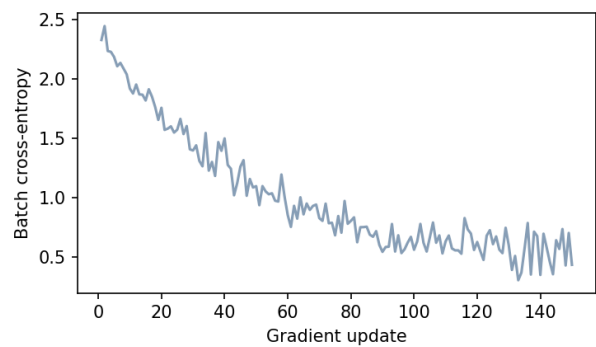
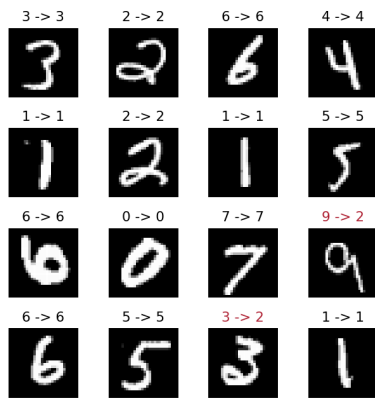


Figure 1: True label → prediction for the first 16 validation images, and the training loss. Incorrect predictions are red.

Exercise

Make a new model with 32 hidden units and train it from fresh weights for the same number of updates. Compare the parameter count, validation accuracy and loss. Next change only the learning rate to 0.001 or 0.01. Keep the random seed, batch size and update count fixed when comparing two choices. Which change helped, and did it help every digit equally?

For the wider network, change `make_model()` to `make_model(width=32)` and rerun the cells from model creation through training and validation.